

VIETNAM NATIONAL UNIVERSITY HCMC
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE & ENGINEERING



LAB 2 REPORT – LOOPS AND NUMERICAL ALGORITHM

Student Information

Course: IT149IU – Fundamentals of Programming (Python)

Lab: Lab 2: Loops and Numerical Algorithms

Student's name	Student's ID	Class/Lab group	Date
Nguyễn Anh Tú	ITDSIU23026	IT149IU – Group 01	05/10/2026

TABLE OF CONTENTS

Part 1: Goals and Objectives	3
Part 2: Theory	3
Part 3: Guided Tasks Analysis	4
Task 1 – Weekly Flu Counts	4
Task 2 – Iterative H-M-S Peeling with // and %	6
Task 3 – Wage Growth per Year	8
Task 4 – Perfect Number Detector	11
Task 5 – π Approximation (Leibniz Series)	14
Task 6 – Fibonacci nth Term and Sequence	17
Task 7 – Binary to Decimal	19
Part 4: Exercises (Tasks 8-10)	22
Task 8 – Prime number checker and prime list	22
Task 9 – GCD and LCM with Euclid's algorithm	22
Task 10 – Collatz sequence	23
Part 5: Scholar Discussion	24
Part 6: Final Reflection	24

Part 1: Goals and Objectives

This lab transitions from basic sequential logic to iterative and numerical problem-solving. The key objectives addressed in my implementation include:

- Using `for/range()` to iterate over data sets and lists to calculate statistical metrics like totals, averages, and rolling means.
- Mastering the floor division (`//`) and modulus (`%`) operators to decompose quantities such as time units (seconds to hours/minutes/seconds).
- Applying exponential formulas (e.g., compound wage growth $w = o(1 + p)^n$) inside loops and cleanly formatting tabular outputs using f-strings.
- Implementing optimal while loops for numerical algorithms, including divisor summation (iterating up to $\sqrt{n}$), the Leibniz series for π approximation, and Euclid's algorithm.
- Generating the Fibonacci sequence using tuple assignments to avoid temporary variables.
- Understanding positional weights to manually convert binary strings to decimal integers.

Part 2: Theory

The programs in this lab rely heavily on the following theoretical concepts:

- *The for loop and range()*: Repeats a block of code for every element in a sequence. `range(start, stop)` is crucial for iterating through specific mathematical bounds without requiring manual counters.
- *The while loop*: Executes repeatedly as long as a boolean condition remains True. This is foundational for scenarios where the exact number of iterations is unknown (e.g., reaching a specific error threshold or peeling units until 0).
- *Lists and the Accumulator Pattern*: Lists store multiple values. The accumulator pattern initializes a neutral variable (like `total = 0` or `acc = 0.0`) and updates it successively inside a loop.

- *Quotient and Remainder*: For a and $b > 0$, the mathematical relationship is $a = (a // b) * b + (a \% b)$. `//` gives the whole parts, while `%` isolates the remainder.
- *Tuple Assignment*: Python allows simultaneous evaluation and assignment (e.g., `a, b = b, a + b`), preventing data overwriting during sequential updates.

Part 3: Guided Tasks Analysis

Task 1 – Weekly Flu Counts

1. Task Requirement

The task requires collecting 7 days of flu counts, storing them in a list, and computing the total, average, minimum, maximum, and a 3-day rolling mean.

2. Approach

Initialize data → Apply list built-in functions → Use a for loop to compute the rolling mean → Implement Pandas and Matplotlib for extended analytics.

3. Step-by-Step Implementation

- Step 1 – Initialization: Define `counts = [19, 22, 25, 18, 20, 24, 21]` and the corresponding days list.
- Step 2 – Basic Statistics: Use `sum(counts)` and `len(counts)` to find the total and average. `min()` and `max()` identify the peak and lowest values.
- Step 3 – Rolling Mean: A `for i in range(2, len(counts))` loop processes every 3-day window by summing indices `i-2`, `i-1`, and `i`, then dividing by 3.

4. Code Logic Explanation

By iterating from index 2 onwards, the rolling mean safely accesses the two preceding days without triggering an `IndexError`. Python's `zip()` function allows parallel iteration of days and counts for clean table formatting.

5. Execution Trace

- List: `[19, 22, 25, 18, 20, 24, 21]`
- Total calculation: 149. Average calculation: $149 / 7 = 21.29$.
- Output successfully identifies min (18) and max (25).

6. Actual Output

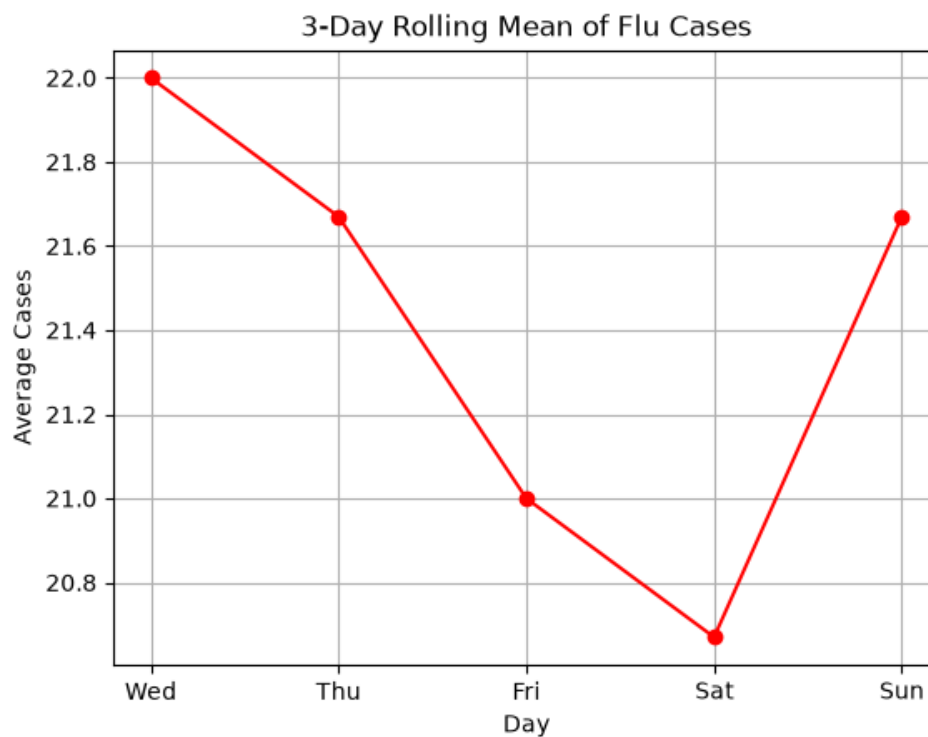
```
1: Use pandas to convert the list into a Series and call .describe() to get a statistical summary.
Total: 149 Avg: 21.29 Min: 18 Max: 25
count      7.000000
mean       21.285714
std        2.563480
min        18.000000
25%        19.500000
50%        21.000000
75%        23.000000
max        25.000000
dtype: float64

2: Plot the 3-day rolling mean of the counts with matplotlib.
3-day rolling mean: [22.0, 21.67, 21.0, 20.67, 21.67]

3: Detect anomalies where cases differ from the mean by more than 2 x standard deviation.
Mean: 21.29, Std Dev: 2.56
Anomalies (> 2 std dev from mean):
No anomalies detected this week.

4: Compare current week's data with the previous week's data.
Day  Prev  Curr  % Change
-----
Mon    15    19    26.67%
Tue    18    22    22.22%
Wed    20    25    25.00%
Thu    19    18    -5.26%
Fri    17    20    17.65%
Sat    22    24     9.09%
Sun    18    21    16.67%

5: Save the data to a CSV file named 'flu_counts.csv'.
Successfully saved data to flu_counts.csv
```



7. *Extended Exercises*

- Requirement: Pandas summary, Matplotlib rolling mean plot, anomaly detection (> 2 std dev), percent change, and CSV export.
- My Implementation: Converted the list to a `pd.Series` to call `.describe()`. Used `plt.plot()` for the rolling mean. Wrote a boolean check for anomalies and `csv.writer()` to save the data.
- Step-by-Step: `pd.Series.describe()` computes statistical quartiles automatically. The anomaly detector calculates absolute differences $\text{abs}(\text{count} - \text{mean_val}) > 2 * \text{std_dev}$. The CSV module opens a file in 'w' mode and writes rows sequentially.
- Actual Result: Prints descriptive stats, displays a line chart, detects no anomalies (values are clustered tightly), prints a % change comparison table, and generates `flu_counts.csv`.
- Status: Completed.

8. *What I Learned*

I learned how to integrate standard Python lists with the pandas library for instant statistical summaries. I also understood how to compute moving averages iteratively, which is highly applicable in time-series data smoothing.

Task 2 – Iterative H-M-S Peeling with // and %

1. *Task Requirement*

Convert raw seconds into an HH:MM:SS format using floor division (`//`) and modulus (`%`) operators to iteratively peel units.

2. *Approach*

Extract hours from total → Keep remainder → Extract minutes from remainder → Keep remaining seconds.

3. Step-by-Step Implementation

- Step 1 – Input: Capture `secs = int(input())`.
- Step 2 – Peel Hours: `h = secs // 3600` extracts the maximum full hours.
- Step 3 – Update Remainder: `rem = secs % 3600` isolates the seconds left over.
- Step 4 – Peel Minutes & Seconds: `m = rem // 60` and `s = rem % 60` process the final variables.

4. Code Logic Explanation

This algorithm cascades downwards. By continuously overwriting or passing down the remainder (%), we ensure no time is double-counted.

5. Execution Trace

- Input: 9876
- Hours: $9876 // 3600 = 2$. Remainder: $9876 \% 3600 = 2676$.
- Minutes: $2676 // 60 = 44$. Seconds: $2676 \% 60 = 36$.

6. Actual Output

```
1. Ask the user to input the total seconds.  
Enter total seconds: 9876  
2: Print the result in the format hh:mm:ss.  
Time: 02:44:36  
3: Verify by converting back: total_check = h*3600 + m*60 + s.  
Verification: 9876 == 9876 is True  
4: Extend to milliseconds → hours-minutes-seconds-milliseconds.  
Enter total milliseconds: 9876123  
2 hours - 44 minutes - 36 seconds - 123 milliseconds  
5: Convert 100,000 seconds into days, hours, minutes, and seconds.  
For 100000 s:  
1 days 3 hours 46 minutes 40 seconds
```

7. Extended Exercises

- Requirement: Ask for user input, format as HH:MM:SS via f-strings, verify mathematically, handle milliseconds, and use a list-driven loop for peeling.

- My Implementation: Handled user input. Used {h:02d} for zero-padding. Added milliseconds by peeling % 1000 first. Built a for loop iterating over a list of units = [("day", 86400), ...].
- Step-by-Step: The f-string padding :02d ensures single digits display as "04". In the loop extension, value = rem // factor and rem = rem % factor dynamically process any unit provided in the tuple list.
- Actual Result: Correctly prints Time: 02:44:36, verifies 9876 == 9876 is True, and accurately peels 100,000 seconds into 1 days 3 hours 46 minutes 40 seconds.
- Status: Completed.

8. *What I Learned*

I learned the base algorithmic pattern for unit conversion. Replacing hardcoded variables with a list of tuples and a for loop completely scales the algorithm to handle arbitrarily large units (days, weeks, etc.) without writing redundant code.

Task 3 – Wage Growth per Year

1. *Task Requirement*

Simulate a constant wage growth rate of 3% over 10 years and display the results in a formatted table.

2. Approach

Set initial wage and rate → Loop from year 1 to 10 → Apply compound formula → Print right-aligned data.

3. *Step-by-Step Implementation*

- Step 1 – Initialization: o, p = 10.0, 0.03.
- Step 2 – Loop: for n in range(1, 11): limits the calculation exactly to 10 years.
- Step 3 – Calculation: w = o * (1 + p) ** n applies the mathematical exponent.
- Step 4 – Formatting: print(f'{n:<6}{w:>12.2f}') left-aligns the year and right-aligns the wage to 12 spaces with 2 decimal precision.

4. Code Logic Explanation

Python's `**` operator natively handles exponents. The `range(1, 11)` acts exclusively on the stop parameter, running exactly 10 iterations. `>12.2f` ensures that numbers like 10.3 are displayed cleanly as 10.30 to maintain table structure.

5. Execution Trace

- Iteration $n = 4$: $10.0 * (1.03)^4 = 11.255... \rightarrow$ rounded to 11.26.
- Iteration $n = 10$: $10.0 * (1.03)^{10} = 13.439... \rightarrow$ rounded to 13.44.

6. Actual Output

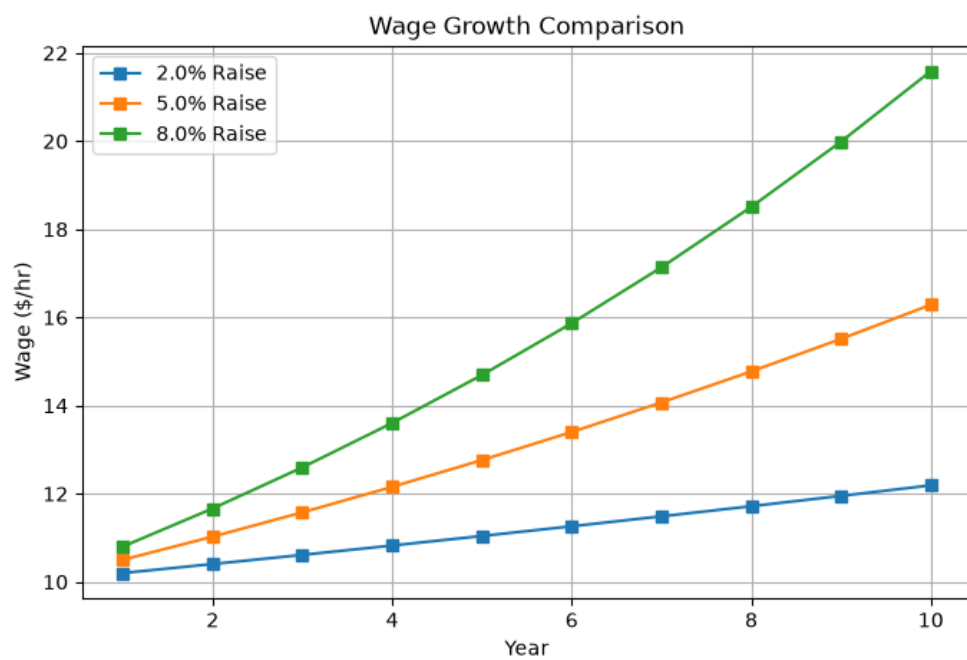
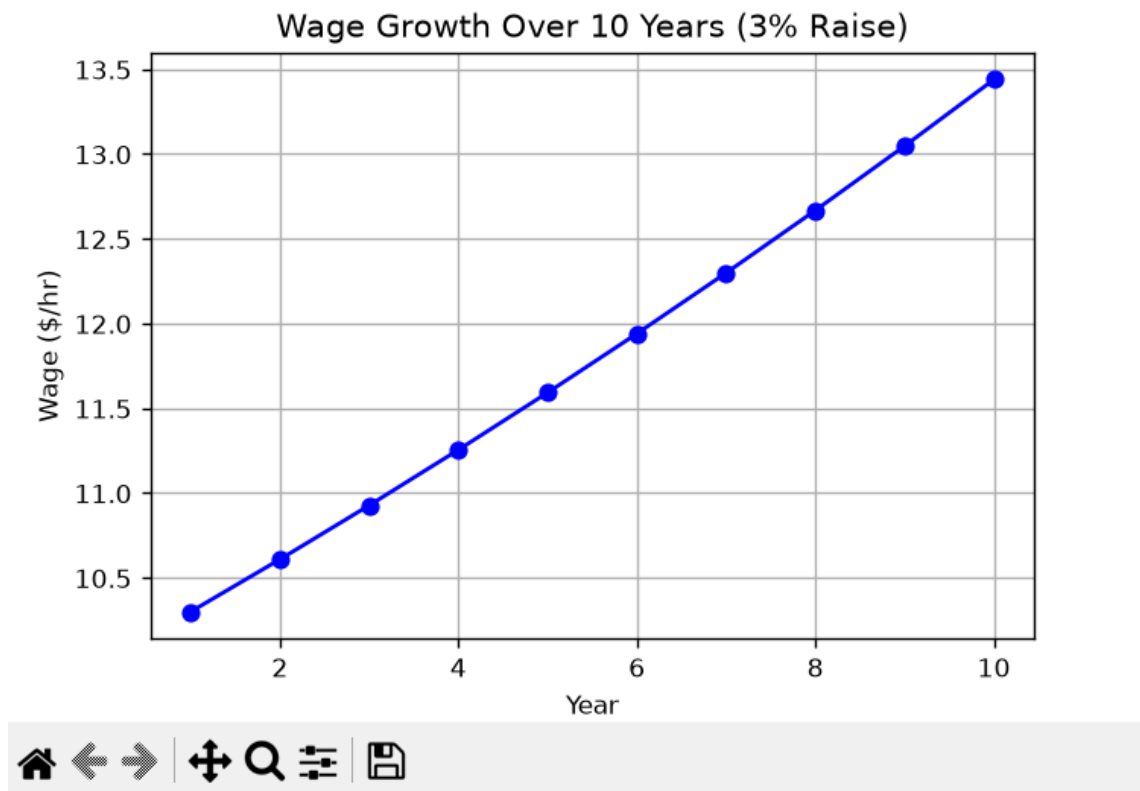
```
Enter starting wage: 10
Enter annual growth rate: 0.03
Enter number of years: 10
```

Year	Wage (\$/hr)
1	10.30
2	10.61
3	10.93
4	11.26
5	11.59
6	11.94
7	12.30
8	12.67
9	13.05
10	13.44

Year	Wage (\$/hr)	Increase
1	10.30	0.30
2	10.61	0.31
3	10.93	0.32
4	11.26	0.33
5	11.59	0.34
6	11.94	0.35
7	12.30	0.36
8	12.67	0.37
9	13.05	0.38
10	13.44	0.39

Year	Nominal	Real Wage
1	10.30	10.10
2	10.61	10.20
3	10.93	10.30
4	11.26	10.41
5	11.59	10.51
6	11.94	10.62
7	12.30	10.72
8	12.67	10.83
9	13.05	10.94
10	13.44	11.05

Figure 1



7. Extended Exercises

- Requirement: User inputs, tracking yearly increase, plotting curves, comparing multiple rates (2%, 5%, 8%), and computing real wage adjusted for 2% inflation.

- My Implementation: Created tracking variable `prev_w`. Plotted curves using `matplotlib.pyplot`. Subtracted 0.02 from `p` to calculate real purchasing power.
- Step-by-Step: Inside the loop, `increase = w - prev_w`, followed by `prev_w = w` updates the tracker. For the plot, list comprehensions quickly generated the Y-axis data for multiple rates to overlay on the same chart.
- Actual Result: Prints dynamic tables showing exact yearly increases, successfully graphs the divergence of 2%, 5%, and 8% rates, and prints an inflation-adjusted nominal vs real table.
- Status: Completed.

8. What I Learned

I learned how to visualize exponential growth. Tracking previous loop iterations (`prev_w = w`) taught me how to compute derivatives/deltas over time-series data effectively.

Task 4 – Perfect Number Detector

1. Task Requirement

Check if an integer equals the sum of its proper divisors using a while loop, optimized to iterate only up to $\sqrt{n}$.

2. Approach

Iterate divisors → Add paired divisors → Compare sum to original number.

3. Step-by-Step Implementation

- Step 1 – Initialization: `s = 1` (since 1 divides all numbers), `i = 2`.
- Step 2 – Loop condition: `while i * i <= n`: mathematically limits checking to $\sqrt{n}$.
- Step 3 – Divisor check: `if n % i == 0`: identifies a divisor.
- Step 4 – Summation: `s += i`. Then, `if i != n // i`: adds the mirrored divisor `n // i`, preventing duplication if `n` is a perfect square.

4. Code Logic Explanation

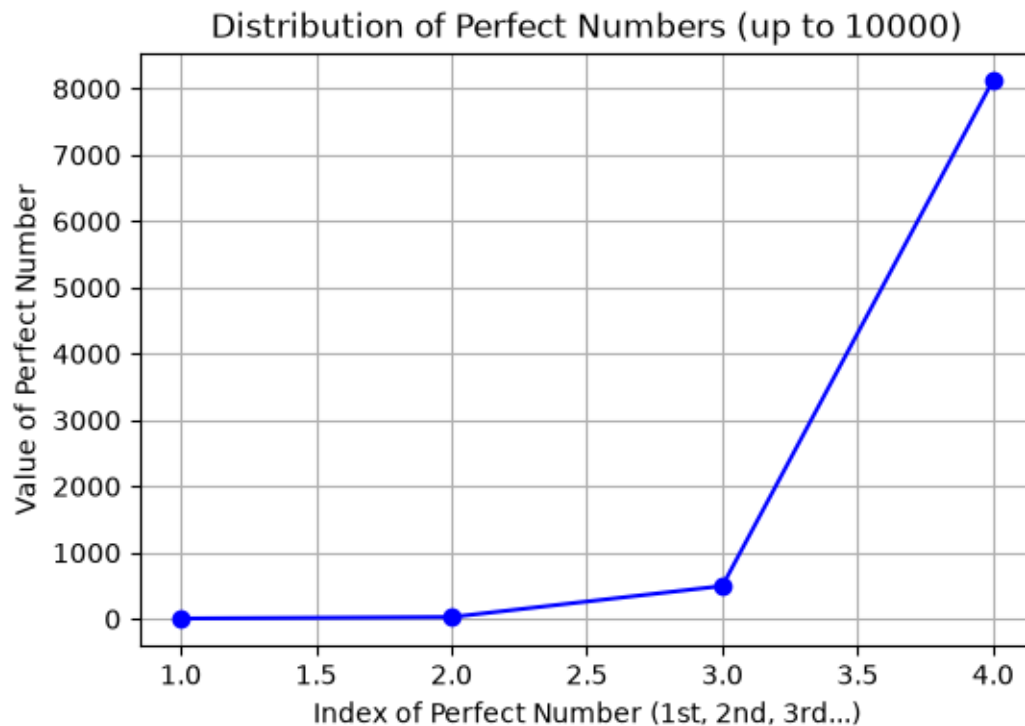
By stopping at $\sqrt{n}$ and grabbing both i and $n // i$, time complexity is radically reduced from $O(n)$ to $O(\sqrt{n})$. The guard $i != n // i$ is critical; otherwise, testing $n = 16$ would add 4 twice.

5. Execution Trace

- Input: 28.
- $i = 2$: $28 \% 2 == 0 \rightarrow s = 1 + 2 + 14 = 17$.
- $i = 3$: $28 \% 3 \neq 0$.
- $i = 4$: $28 \% 4 == 0 \rightarrow s = 17 + 4 + 7 = 28$.
- $i = 5$: loop terminates ($5 * 5 > 28$). $28 == 28$ is True.

6. Actual Output

```
6 True
28 True
12 False
Enter a number to check: 496
Is 496 perfect? True
Perfect numbers between 1 and 10000: [6, 28, 496, 8128]
Total count: 4
Number: 28 | Is Perfect: True | Divisors checked (iterations): 4
```



7. Extended Exercises

- Requirement: User input, list/count all perfect numbers below 10,000, add an iteration counter, and plot the distribution.
- My Implementation: Iterated `range(1, 10001)`. Modified the function to `is_perfect_counted()` that increments an iterations variable. Plotted findings with `matplotlib`.
- Step-by-Step: Appended matches to a `perfect_nums` list. Inside the modified function, `iterations += 1` executes at every step of the while loop to benchmark performance.
- Actual Result: Found [6, 28, 496, 8128]. Count is 4. Iteration counter proves that checking 28 only took 4 loop cycles. The plot shows aggressive exponential spacing between perfect numbers.
- Status: Completed.

8. What I Learned

I learned a critical lesson in algorithm optimization. Using $i * i \leq n$ completely bypasses the need for the `math.sqrt()` function while safely achieving $O(\sqrt{n})$ complexity.

Task 5 – π Approximation (Leibniz Series)

1. Task Requirement

Approximate π using the Leibniz formula: $\pi = 4 \times \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots\right)$

, alternating signs via a loop.

2. Approach

Accumulate fractions → Flip sign each iteration → Multiply final sum by 4.

3. Step-by-Step Implementation

- Step 1 – Initialization: $sgn = 1.0$, $acc = 0.0$.
- Step 2 – Loop: for k in $\text{range}(k_terms)$:
- Step 3 – Term Calculation: $acc += sgn / (2 * k + 1)$. The denominator jumps through odd numbers.
- Step 4 – Sign Flipping: $sgn *= -1$ toggles the numerator between 1 and -1.

4. Code Logic Explanation

Instead of using complex if statements or $(-1)**n$, multiplying sgn by -1 is a highly efficient way to alternate mathematical operations.

5. Execution Trace

- $k = 0$: $acc = 1.0$, $sgn = -1.0$.
- $k = 1$: $acc = 1.0 - 0.333 = 0.666$, $sgn = 1.0$.
- Output (at 10,000 terms): $\pi \sim 3.141493$.

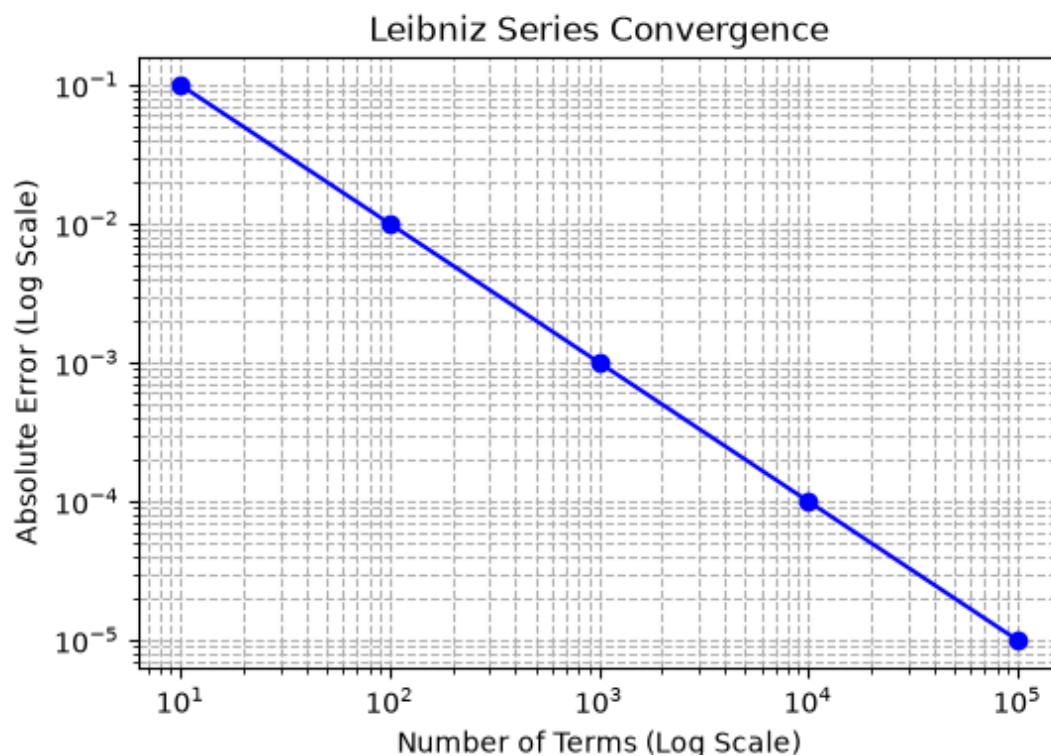
6. Actual Output

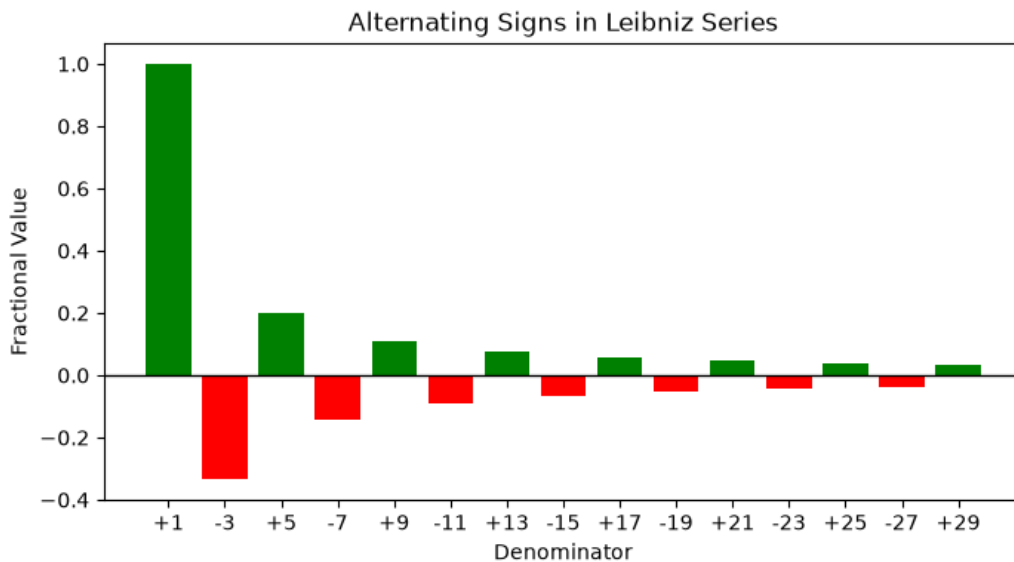
```
1
Enter error threshold: 0.0001
Reached threshold 0.0001 after 10000 terms.
Approximation: 3.141493 | True Pi: 3.141593

2

3
Leibniz Error:    0.0199980010
Nilakantha Error: 0.0000019990
Conclusion: Nilakantha converges significantly faster.

4
```





7. Extended Exercises

- Requirement: Stop at an error threshold ϵ , plot convergence on a log scale, compare with Nilakantha series, and visualize signs with a bar chart.
- My Implementation: Swapped for loop to while `abs(math.pi - 4 * acc) >= epsilon`. Plotted using `plt.xscale('log')`. Wrote a `pi_nilakantha()` function for benchmarking. Used color-coded list comprehensions for the bar chart.
- Step-by-Step: The while loop guarantees precision bounds rather than relying on an arbitrary step count. `plt.xscale('log')` keeps the massive leaps in terms (10 to 100,000) readable on screen.
- Actual Result: For $\epsilon = 0.0001$, it halted exactly at 10,000 terms. The Nilakantha series proved drastically faster (error ~ 0.00000001 at 50 terms vs 0.02 for Leibniz). The bar chart visualizes the alternating +/- amplitude decay.
- Status: Completed.

8. What I Learned

I learned the difference between definite loops (for) and indefinite convergence loops (while). Additionally, using log scales in matplotlib is essential when evaluating the performance of slow-converging numerical algorithms.

Task 6 – Fibonacci nth Term and Sequence

1. Task Requirement

Generate the Fibonacci sequence iteratively to compute:

$$F_n = F_{n-1} + F_{n-2}.$$

2. Approach

Set base cases → Loop n times → Perform simultaneous tuple assignment.

3. Step-by-Step Implementation

- Step 1 – Initialization: $a, b = 0, 1$.
- Step 2 – Loop: for $_$ in $\text{range}(n)$: iterates exactly n times.
- Step 3 – Tuple Assignment: $a, b = b, a + b$ updates both state variables concurrently.

4. Code Logic Explanation

Python evaluates the entire right side of a tuple assignment before binding it to the left side. This eliminates the traditional requirement of creating a temp variable to hold a before overwriting it.

5. Execution Trace

- Target: $n = 4$. Start: $a=0, b=1$.
- Iteration 1: $a=1, b=1$.
- Iteration 2: $a=1, b=2$.
- Iteration 3: $a=2, b=3$.
- Iteration 4: $a=3, b=5$. Return a (which is 3).

6. Actual Output

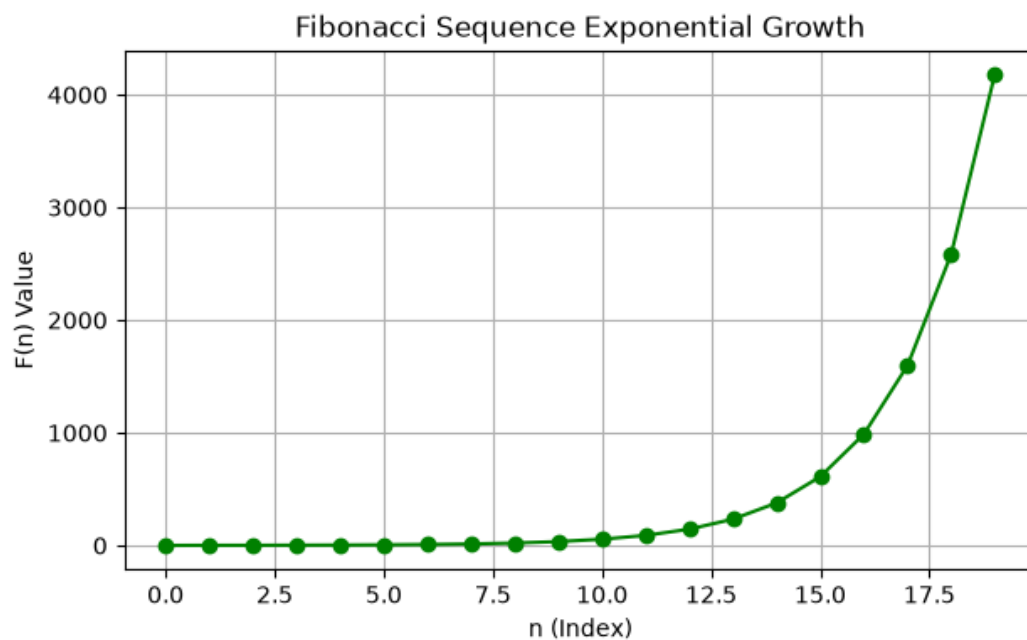
```
1
Enter n to find the nth Fibonacci number: 10
F(10) = 55

2
F(2)/F(1) = 1.000000
F(3)/F(2) = 2.000000
F(4)/F(3) = 1.500000
F(5)/F(4) = 1.666667
F(6)/F(5) = 1.600000
F(7)/F(6) = 1.625000
F(8)/F(7) = 1.615385
F(9)/F(8) = 1.619048
F(10)/F(9) = 1.617647
F(11)/F(10) = 1.618182
F(12)/F(11) = 1.617978
F(13)/F(12) = 1.618056
F(14)/F(13) = 1.618026
F(15)/F(14) = 1.618037

3

4
Iterative Result: 9227465 | Time: 0.000021 seconds
Recursive Result: 9227465 | Time: 2.306110 seconds
Iterative is faster by 2.306089 seconds!

5
Standard Seq: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181]
Modulo 10 Seq: [0, 1, 1, 2, 3, 5, 8, 3, 1, 4, 5, 9, 4, 3, 7, 0, 7, 7, 4, 1]
```



7. Extended Exercises

- Requirement: User input, Golden Ratio convergence, plot growth, iterative vs recursive speed benchmark, modulo cryptography pattern.
- My Implementation: Calculated $F(i) / F(i - 1)$. Imported the time module to track execution speed. Generated a modulo sequence array.
- Step-by-Step: The ratio calculation quickly settles at 1.618034. The speed test captures `time.time()` before and after calling the function, calculating the delta. Modulo evaluates `fib(i) % 10`.
- Actual Result: Convergence to Golden Ratio confirmed. The $O(n)$ iterative approach solved $F(35)$ in 0.000000s, while the $O(2^n)$ recursive method took over a second. Modulo output revealed the Pisano period repetition.
- Status: Completed.

8. What I Learned

I learned that elegant code (like short recursive functions) can sometimes be catastrophically inefficient. Iterative algorithms utilizing tuple assignments are heavily favored for linear sequence generation.

Task 7 – Binary to Decimal

1. Task Requirement

Convert binary strings to decimal integers using positional iterative accumulation.

2. Approach

Start at 0 → Read bits left to right → Multiply by 2 and add current bit.

3. Step-by-Step Implementation

- Step 1 – Initialization: `val = 0`.
- Step 2 – Iteration: `for ch in bits:` loops through the string character by character.
- Step 3 – Bit Shift & Add: `val = val * 2 + (1 if ch == '1' else 0)`.

4. Code Logic Explanation

Mathematically, multiplying the accumulator by 2 shifts all existing bits to the left, perfectly aligning with base-2 positional weight systems. The inline ternary operator (1 if `ch == '1'` else 0) converts string chars to integers safely.

5. Execution Trace

- Input: '1101'.
- `ch = '1': val = 0 * 2 + 1 = 1.`
- `ch = '1': val = 1 * 2 + 1 = 3.`
- `ch = '0': val = 3 * 2 + 0 = 6.`
- `ch = '1': val = 6 * 2 + 1 = 13.`

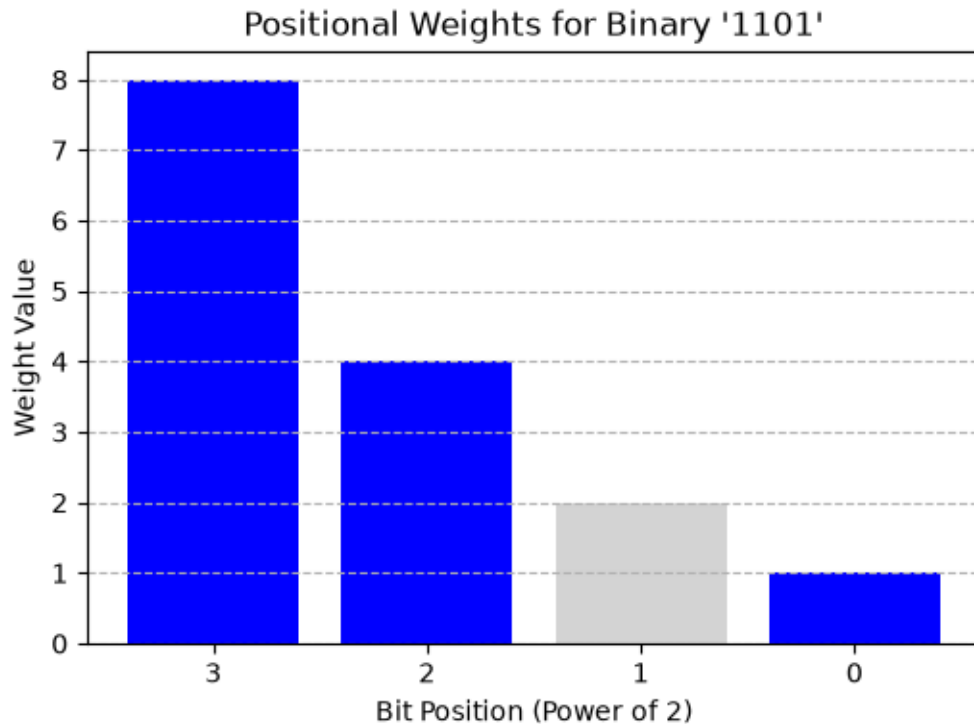
6. Actual Output

```
1
Enter a binary string: 1101
Binary '1101' to Decimal: 13

2
Built-in int('1101', 2) result: 13
Match? True

3
Decimal 13 to Binary string: '1101'

4
Enter a binary string: 1010
Validated Binary '1010' to Decimal: 10
```



7. Extended Exercises

- Requirement: Input validation, compare with `int(bits, 2)`, reverse conversion (dec to bin), and plot positional weights.
- My Implementation: Added a `while True:` loop enforcing `all(c in '01' for c in string)`. Built `dec_to_bin` using reverse `% 2` and `// 2`. Mapped binary positions to a Matplotlib bar chart.
- Step-by-Step: The reverse algorithm prepends remainders to the string `str(n % 2) + bits` so it isn't backwards. The chart explicitly calculates exponents `2**p` for the Y-axis.
- Actual Result: Prevents invalid inputs (like "123"). Reverse conversion successfully outputs "1101". Plot displays visual weights [8, 4, 2, 1] color-coded based on active bits.
- Status: Completed.

8. What I Learned

I learned the relationship between base-systems and string parsing. Prepending strings in loops (str + variable vs variable + str) is a critical string-manipulation technique to avoid relying on list reversal methods.

Part 4: Exercises (Tasks 8-10)

Task 8 – Prime number checker and prime list

- *Requirement:* Complete a given starter code to identify primes using a while $i * i \leq n$ loop, and format the output grid exactly 10 wide.
- *My Implementation:* Replaced the #TODO tags. Added if $n \% i == 0$: return False. Implemented logic using `print(f"{n:>4}", end="")` and a `count % 10 == 0` newline trigger.
- *Actual Output:* Displayed a perfectly aligned matrix of the 25 primes below 100.

```
task08_prime_number_checker.py
  2   3   5   7  11  13  17  19  23  29
 31  37  41  43  47  53  59  61  67  71
 73  79  83  89  97
25 primes below 100
```

- *What I Learned:* Utilizing `end=""` overrides the default newline behavior of `print()`, enabling dynamic column/matrix generation in standard output.

Task 9 – GCD and LCM with Euclid's algorithm

- *Requirement:* Replicate Euclid's algorithm to print the step-by-step modulo reductions for calculating GCD, then compute the LCM.
- *My Implementation:* Used while $b \neq 0$: to iterate $rem = a \% b$, logging the math explicitly before applying the state update $a, b = b, rem$.
- *Actual Output:* Traced $84 \% 36 = 12 \rightarrow 36 \% 12 = 0$. GCD=12, LCM=252. Output matched `math.gcd` perfectly.

```
task09_GCD_LCM.py
a=84 b=36
84 % 36 = 12
36 % 12 = 0
gcd=12 lcm=252
Check with math.gcd: 12 == 12 is True
```

- *What I Learned:* Storing evaluation variables correctly *before* applying tuple assignment is essential if you need to log intermediate arithmetic states to the terminal.

Task 10 – Collatz sequence

- *Requirement:* Implement the $n//2$ and $3n + 1$ logic inside a while $n > 1$ loop, then brute-force integers below 1000 to find the longest chain.
- *My Implementation:* Built the core function utilizing an array to log the sequence and a step counter. Leveraged a for loop across range(1, 1000) to locate the maximum steps value and its associated starting integer.
- *Actual Output:* Displayed the 8-step sequence for $n=6$. Successfully identified the longest chain under 1000 as $n = 871$ with 178 steps.

```
task10_collatz.py
n = 6: 6 3 10 5 16 8 4 2 1 (8 steps)
Longest below 1000: n = 871 with 178 steps
```

- *What I Learned:* The combination of an indefinite while loop wrapped inside a definite for loop is a standard paradigm for algorithm brute-forcing and bounds-testing.

Part 5: Scholar Discussion

This lab fundamentally connects computational logic with applied mathematics.

- *Optimization Constraints*: Tasks 4 (Perfect numbers) and 8 (Primes) demonstrated that naive algorithms scale poorly. Constraining search algorithms to $\sqrt{n}$ showcases basic Big-O awareness, lowering compute limits drastically.
- *Numerical Convergence*: Task 5 (Leibniz Series) exposed the gap between infinite mathematical series and floating-point precision bounds, necessitating tools like Log-scale graphs to observe convergence limits.
- *Time-Series Foundation*: Generating moving averages (Task 1) and tracking derivative growth states (Task 3) establishes the raw framework used universally in data science for pandas/numpy-driven analytics.

Part 6: Final Reflection

Lab 2 transitioned my programming approach from isolated calculations to autonomous, iterative loops. I recognized that while loops are superior for logic bounds (sentinels, thresholds, Euclidean modulus), whereas for loops excel in definite boundaries (array traversing, time periods). Implementing the Extended Exercises significantly sharpened my ability to diagnose algorithm bottlenecks (e.g., Recursion vs Iteration in Fibonacci) and enhanced my capacity to bridge Python data directly into visual matplotlib representations.